Products, R&D & Capacity — Authored Content

Full content for the depth system across all **6 playable sub-industries**, authored against the *Products, R&D & Capacity: Authoring Spec*. Four blocks per sub-industry: R&D lines, product archetypes, capacity types, and a per-industry tuning block.

Drop-in paths:

- `content/rd_lines/<sub_industry>.toml`
- `content/products/<sub_industry>.toml`
- `content/capacity/<sub_industry>.toml`
- `content/products/_tuning.toml` (all six blocks, one file)

Validation summary

All content passes the spec's sanity checklist:

- Every product `gates` references a real R&D line id in the same sub-industry.
- Every `capacity_type` references a real capacity id in the same sub-industry.
- Every `specs` weight vector sums to exactly **1.0**.
- Every `specs` tag traces to a line's `drives_specs`.
- Every capacity type has 6 monotonically increasing rungs (never caps out).
- All six `_tuning.toml` blocks present and complete.
- Tier progressions are monotonic (cheap/fast beachhead -> expensive/slow bet-the-company).
- **No sub-industry is strictly easiest** — each is hard on at least one difficulty axis, spread across different axes so each plays distinctly.

Per-industry contrast at a glance:

sub_industry	cost×	time×	decay×	parallel	volatility	hard axis

frontier_model_lab	1.0	1.0	1.0	4	0.25	churn + hype volatility
vertical_ai_saas	0.55	0.6	0.7	3	0.10	winning share vs. incumbents
ai_chips	1.4	1.6	1.1	3	0.15	slow / expensive / few parallel
launch_services	1.2	0.7	0.6	3	0.12	capital-heavy, few parallel
satellite_constellations	1.3	1.2	0.8	3	0.16	capital buildout, two-capacity juggle
space_stations	1.5	1.7	0.5	2	0.18	slowest / costliest / fewest bets

---

=== ai_chips ===

content/rd_lines/ai_chips.toml

```
# content/rd_lines/ai_chips.toml
# 3 lines that pull in different directions: chase the node (expensive, the
# biggest lever) vs. bet on architecture (cheaper, directional) vs. perfect
# yield (cheapest, protects margin). You can't fully fund all three.

[process_node]
id = "process_node"
sub_industry = "ai_chips"
name = "Process node"
description = """How small you can etch. Each step down — 5nm, 3nm, 2nm — packs m
compute per watt and is the single biggest driver of whether your silicon is \
competitive. Brutally expensive, slow, and the whole industry is sprinting."""
```

```
icon = "chip"
starting_level = 20
base_cost_per_quarter = 8
drives_specs = ["performance", "efficiency"]

[architecture]
id = "architecture"
sub_industry = "ai_chips"
name = "Architecture"
description = """Your core design — how the chip is laid out and what it's good a
A clever architecture can beat a smaller node. This is where you bet on a \
direction (general-purpose vs. AI-specialized) the market may or may not reward."
icon = "blueprint"
starting_level = 25
base_cost_per_quarter = 6
drives_specs = ["performance", "specialization"]

[yield_line]
id = "yield_line"
sub_industry = "ai_chips"
name = "Yield & packaging"
description = """How many good chips you get per wafer, and how you package them.
Unglamorous, but yield is margin. Low yield means you're scrapping silicon and \
bleeding cash on every batch."""
icon = "stack"
starting_level = 30
base_cost_per_quarter = 4
drives_specs = ["margin", "cost"]
```

content/products/ai_chips.toml

```
# content/products/ai_chips.toml
# tier 1 -> 3 progression: cheap/fast/low-margin beachhead -> bet-the-company par
# specs are WEIGHTS (sum ~1.0) mapping R&D lines -> product quality.

[consumer_gpu]
id = "consumer_gpu"
sub_industry = "ai_chips"
name = "Consumer GPU"
tier = 1
description = """A graphics card for gamers and creators. Your entry product — \
lower margins, huge volume, and a brand-building beachhead. The training wheels \
of silicon: it teaches your fab to walk before you bet the company on datacenter
flavor_naming_hint = "e.g. 'GX-1', 'Vega', a number+letter consumer brand"
```

```
[consumer_gpu.gates]
process_node = 15
architecture = 20

[consumer_gpu.economics]
build_cost = 40
build_weeks = 60
unit_margin = 0.35
capacity_type = "fab_line"
capacity_to_build = 1
capacity_to_run = 1
addressable_market = 1200
ramp_weeks = 26
decay_per_quarter = 0.04

[consumer_gpu.specs]
performance = 0.5
efficiency = 0.3
margin = 0.2

[datacenter_accel]
id = "datacenter_accel"
sub_industry = "ai_chips"
name = "Datacenter accelerator"
tier = 2
description = """The product that makes empires. Sold by the rack to AI labs and
clouds, priced like jewelry, and the thing OpenMind and the hyperscalers are \
desperate for. Demands a real process node and flawless yield — and the AI-hype \
cycle can make or unmake it overnight."""
flavor_naming_hint = "e.g. 'H-series', 'Hopper-class', a datacenter codename"

[datacenter_accel.gates]
process_node = 45
architecture = 40
yield_line = 35

[datacenter_accel.economics]
build_cost = 220
build_weeks = 130
unit_margin = 0.62
capacity_type = "fab_line"
capacity_to_build = 2
capacity_to_run = 2
addressable_market = 9000
ramp_weeks = 39
decay_per_quarter = 0.09
```

```
[datacenter_accel.specs]
performance = 0.55
efficiency = 0.30
margin = 0.15

[next_gen_arch]
id = "next_gen_arch"
sub_industry = "ai_chips"
name = "Next-gen architecture part"
tier = 3
description = """A generational leap — a part that redefines the category and \
locks in customers for years. Requires bleeding-edge everything and a fortune, \
but if it lands you set the market's pace and the rivals chase you."""
flavor_naming_hint = "e.g. a bold codename — 'Helios', 'Titan'"

[next_gen_arch.gates]
process_node = 70
architecture = 65
yield_line = 55

[next_gen_arch.economics]
build_cost = 600
build_weeks = 180
unit_margin = 0.68
capacity_type = "fab_line"
capacity_to_build = 3
capacity_to_run = 2
addressable_market = 16000
ramp_weeks = 52
decay_per_quarter = 0.10

[next_gen_arch.specs]
performance = 0.6
efficiency = 0.25
margin = 0.15
```

content/capacity/ai_chips.toml

```
# content/capacity/ai_chips.toml
# one capacity type (fab lines), 6 rungs so it never caps out.

[fab_line]
id = "fab_line"
sub_industry = "ai_chips"
name = "Fab line"
```

```
unit_label = "line"
description = """Where silicon gets made. Each line is an enormous capital projec
that takes time to stand up — and your products can't ship or scale without one.
Run out of lines and your best chip just sits in a queue."""

[[fab_line.rungs]]
capacity = 2
cost = 60
build_weeks = 52
[[fab_line.rungs]]
capacity = 4
cost = 140
build_weeks = 60
[[fab_line.rungs]]
capacity = 7
cost = 300
build_weeks = 72
[[fab_line.rungs]]
capacity = 11
cost = 560
build_weeks = 84
[[fab_line.rungs]]
capacity = 16
cost = 980
build_weeks = 96
[[fab_line.rungs]]
capacity = 22
cost = 1600
build_weeks = 108
```

=== frontier_model_lab ===

content/rd_lines/frontier_model_lab.toml

```
# content/rd_lines/frontier_model_lab.toml
# 3 lines pulling apart: raw scaling (compute+params, the headline driver) vs.
# data quality (cheaper edge, often underrated) vs. alignment/safety (slows raw
# capability but unlocks enterprise trust and dodges the safety-incident events).

[scaling]
id = "scaling"
sub_industry = "frontier_model_lab"
name = "Scaling & pretraining"
description = """How big and how well you train the base model. The headline leve
```

```
— more compute, more parameters, better training recipes. It moves benchmarks \
and press fastest, and it's where the money and the bragging rights go."""
icon = "trend-up"
starting_level = 25
base_cost_per_quarter = 7
drives_specs = ["capability", "benchmark"]

[data_quality]
id = "data_quality"
sub_industry = "frontier_model_lab"
name = "Data & post-training"
description = """The curation, fine-tuning, and RLHF that turn a raw model into a
useful one. Cheaper than brute scale and frequently the real edge — a smaller \
model with great data can beat a bigger sloppy one where it actually counts."""
icon = "database"
starting_level = 30
base_cost_per_quarter = 5
drives_specs = ["capability", "reliability"]

[alignment]
id = "alignment"
sub_industry = "frontier_model_lab"
name = "Alignment & safety"
description = """Making the model behave — refusing what it should, staying \
reliable under pressure. It taxes raw capability progress, but it's what wins \
regulated enterprise customers and keeps you out of the safety-incident headlines
icon = "shield"
starting_level = 20
base_cost_per_quarter = 4
drives_specs = ["reliability", "safety"]
```

content/products/frontier_model_lab.toml

```
# content/products/frontier_model_lab.toml
# 4-archetype progression. Note the contrast with chips: build_weeks are far
# shorter, build_costs lower, decay HIGHER (the SOTA treadmill). A frontier lab
# ships often and obsoletes fast — the opposite feel to silicon.

[api_model]
id = "api_model"
sub_industry = "frontier_model_lab"
name = "API model endpoint"
tier = 1
description = """A model you serve over an API by the token. Your beachhead — \
developers plug in, you meter usage, and you learn what people actually want \
```

```
before betting big. Low margin per call, but it scales the instant you ship and \
it's how the world first meets your lab."""
flavor_naming_hint = "e.g. 'Helio-mini', a version-numbered API name"

[api_model.gates]
scaling = 20
data_quality = 25

[api_model.economics]
build_cost = 18
build_weeks = 26                # ~6 months — fast cadence vs. chips' years
unit_margin = 0.45
capacity_type = "compute"
capacity_to_build = 1
capacity_to_run = 1
addressable_market = 900
ramp_weeks = 16
decay_per_quarter = 0.12        # models obsolesce fast — the SOTA clock

[api_model.specs]
capability = 0.5
reliability = 0.3
benchmark = 0.2

[flagship_model]
id = "flagship_model"
sub_industry = "frontier_model_lab"
name = "Flagship frontier model"
tier = 2
description = """The big one — the model that defines your lab and tops the \
leaderboard you want to own. A real training run with real compute behind it, \
the kind that draws press, talent, and term sheets when it lands. The AI-hype \
cycle can make it a sensation or a footnote overnight."""
flavor_naming_hint = "e.g. 'Helio-4', a flagship generation name"

[flagship_model.gates]
scaling = 50
data_quality = 45
alignment = 30

[flagship_model.economics]
build_cost = 90
build_weeks = 52                # ~1 year — the headline training run
unit_margin = 0.58
capacity_type = "compute"
capacity_to_build = 3
capacity_to_run = 2
```

```
addressable_market = 7000
ramp_weeks = 20
decay_per_quarter = 0.16        # the frontier treadmill at full speed

[flagship_model.specs]
capability = 0.55
benchmark = 0.25
reliability = 0.20

[enterprise_suite]
id = "enterprise_suite"
sub_industry = "frontier_model_lab"
name = "Enterprise model suite"
tier = 3
description = """A trust-grade deployment for regulated industries — the same \
intelligence wrapped in reliability, compliance, and uptime guarantees. Lower \
hype, stickier revenue: NovaMind built a business on exactly this. Demands real \
alignment work, and rewards it with customers who don't churn on the next \
benchmark."""
flavor_naming_hint = "e.g. 'Helio Enterprise', a trust-forward brand"

[enterprise_suite.gates]
scaling = 55
data_quality = 55
alignment = 60

[enterprise_suite.economics]
build_cost = 120
build_weeks = 60
unit_margin = 0.66
capacity_type = "compute"
capacity_to_build = 2
capacity_to_run = 3             # heavier always-on serving footprint
addressable_market = 8500
ramp_weeks = 39                 # slower enterprise sales cycle
decay_per_quarter = 0.07        # trust products age slowly — the stable anchor

[enterprise_suite.specs]
reliability = 0.45
capability = 0.30
safety = 0.25

[agentic_platform]
id = "agentic_platform"
sub_industry = "frontier_model_lab"
name = "Agentic platform"
tier = 4
```

```
description = """The frontier of the frontier — models that act, not just answer:
planning, using tools, completing real work autonomously. The category-defining \
bet that needs everything firing at once. If it lands you're the platform \
everyone builds on; if alignment lags, it's also the riskiest thing you've shippe
flavor_naming_hint = "e.g. 'Helio Agents', a bold platform name"

[agentic_platform.gates]
scaling = 70
data_quality = 65
alignment = 70

[agentic_platform.economics]
build_cost = 200
build_weeks = 78
unit_margin = 0.64
capacity_type = "compute"
capacity_to_build = 4
capacity_to_run = 4
addressable_market = 18000
ramp_weeks = 30
decay_per_quarter = 0.14

[agentic_platform.specs]
capability = 0.45
reliability = 0.30
safety = 0.25
```

content/capacity/frontier_model_lab.toml

```
# content/capacity/frontier_model_lab.toml
# one capacity type (compute), 7 rungs: pods -> clusters -> datacenters.
# cheaper early rungs than fab lines, faster to stand up — compute scales more
# fluidly than silicon fabs, matching the fast-cadence feel of the sub-industry.

[compute]
id = "compute"
sub_industry = "frontier_model_lab"
name = "Compute"
unit_label = "cluster"
description = """The GPUs and pods your models train and serve on. Every training
run and every live endpoint draws on it, and the frontier is an arms race for \
more. Run short and your next model waits in the allocation queue behind everyone
else's."""

[[compute.rungs]]
```

```
capacity = 3
cost = 30
build_weeks = 20
[[compute.rungs]]
capacity = 6
cost = 75
build_weeks = 26
[[compute.rungs]]
capacity = 10
cost = 160
build_weeks = 32
[[compute.rungs]]
capacity = 16
cost = 320
build_weeks = 38
[[compute.rungs]]
capacity = 24
cost = 600
build_weeks = 44
[[compute.rungs]]
capacity = 34
cost = 1050
build_weeks = 50
[[compute.rungs]]
capacity = 46
cost = 1750
build_weeks = 56
```

=== vertical_ai_saas ===

content/rd_lines/vertical_ai_saas.toml

```
# content/rd_lines/vertical_ai_saas.toml
# 3 lines: domain depth (the vertical moat) vs. platform/integration (stickiness)
# vs. model leverage (riding foundation models without being eaten by them).

[domain_depth]
id = "domain_depth"
sub_industry = "vertical_ai_saas"
name = "Domain expertise"
description = """How deeply your product understands its industry — the workflows
the edge cases, the regulations a generic model never learns. This is the moat: \
the deeper it goes, the harder a horizontal foundation model can swallow you."""
icon = "book"
```

```
starting_level = 30
base_cost_per_quarter = 4
drives_specs = ["fit", "moat"]

[platform]
id = "platform"
sub_industry = "vertical_ai_saas"
name = "Platform & integrations"
description = """The connectors, data pipelines, and admin tooling that wire you
into a customer's stack. Unglamorous plumbing, but it's what makes you sticky — \
once you're embedded in their systems, ripping you out is a project nobody wants.
icon = "plug"
starting_level = 25
base_cost_per_quarter = 5
drives_specs = ["stickiness", "reliability"]

[model_leverage]
id = "model_leverage"
sub_industry = "vertical_ai_saas"
name = "Model leverage"
description = """How well you harness foundation models without being commoditize
by them — orchestration, retrieval, and the thin proprietary layer on top. Get \
it right and every frontier advance is your tailwind; get it wrong and the next \
model release eats your feature."""
icon = "layers"
starting_level = 25
base_cost_per_quarter = 4
drives_specs = ["fit", "reliability"]
```

content/products/vertical_ai_saas.toml

```
# content/products/vertical_ai_saas.toml
# SaaS feel: cheap+fast builds, LOW decay (sticky subscriptions), capacity-light
# (deployment infra is cheap). The strategic pressure here is go-to-market and
# retention, not capital — products are workhorses that compound if you keep them

[point_solution]
id = "point_solution"
sub_industry = "vertical_ai_saas"
name = "Point solution"
tier = 1
description = """A single sharp tool that nails one painful workflow — contract \
review, claims triage, whatever your vertical bleeds time on. Easy to sell \
because the value is obvious, and it gets your foot in the door. The wedge you \
expand from."""
```

```
flavor_naming_hint = "e.g. 'ClauseCheck', a workflow-named tool"

[point_solution.gates]
domain_depth = 20
model_leverage = 15

[point_solution.economics]
build_cost = 8
build_weeks = 18
unit_margin = 0.72              # software margins
capacity_type = "deployment"
capacity_to_build = 1
capacity_to_run = 1
addressable_market = 600
ramp_weeks = 20
decay_per_quarter = 0.05        # sticky once adopted

[point_solution.specs]
fit = 0.5
reliability = 0.3
stickiness = 0.2

[workflow_suite]
id = "workflow_suite"
sub_industry = "vertical_ai_saas"
name = "Workflow suite"
tier = 2
description = """Several connected tools that own an entire department's workflow
end to end. Now you're not a feature, you're the system of record — and Lexiq's \
playbook shows where that leads: 128% net revenue retention because customers \
keep expanding into you."""
flavor_naming_hint = "e.g. 'LegalOS', a suite/platform name"

[workflow_suite.gates]
domain_depth = 40
platform = 40
model_leverage = 30

[workflow_suite.economics]
build_cost = 28
build_weeks = 34
unit_margin = 0.76
capacity_type = "deployment"
capacity_to_build = 1
capacity_to_run = 2
addressable_market = 3200
ramp_weeks = 30
```

```
decay_per_quarter = 0.04        # the workhorse — compounds for years

[workflow_suite.specs]
fit = 0.4
stickiness = 0.35
reliability = 0.25

[platform_play]
id = "platform_play"
sub_industry = "vertical_ai_saas"
name = "Vertical platform"
tier = 3
description = """The category-defining bet: an extensible platform others build \
on top of, with your data network effect deepening every year. Hard to reach — \
it needs real domain depth and rock-solid integrations — but once you're the \
platform, you set the standard and the switching cost becomes almost total."""
flavor_naming_hint = "e.g. 'Lexiq Platform', a platform brand"

[platform_play.gates]
domain_depth = 65
platform = 60
model_leverage = 50

[platform_play.economics]
build_cost = 70
build_weeks = 52
unit_margin = 0.80
capacity_type = "deployment"
capacity_to_build = 2
capacity_to_run = 3
addressable_market = 9000
ramp_weeks = 44
decay_per_quarter = 0.03        # platforms age slowest of all

[platform_play.specs]
moat = 0.4
stickiness = 0.35
fit = 0.25
```

content/capacity/vertical_ai_saas.toml

```
# content/capacity/vertical_ai_saas.toml
# one capacity type (deployment/infra), deliberately CHEAP and fast — SaaS is
# capacity-light per the spec. The constraint here is meant to bite far less
# than fab lines or compute; the real game is go-to-market, not capital.
```

```
[deployment]
id = "deployment"
sub_industry = "vertical_ai_saas"
name = "Deployment infrastructure"
unit_label = "environment"
description = """The cloud infra, security, and customer environments your produc
runs in. Lighter than a fab or a GPU cluster — but each enterprise customer wants
isolation, compliance, and headroom, so growth still costs something. Cheap to \
add relative to hardware businesses."""

[[deployment.rungs]]
capacity = 4
cost = 6
build_weeks = 8
[[deployment.rungs]]
capacity = 8
cost = 15
build_weeks = 10
[[deployment.rungs]]
capacity = 14
cost = 34
build_weeks = 12
[[deployment.rungs]]
capacity = 22
cost = 68
build_weeks = 14
[[deployment.rungs]]
capacity = 32
cost = 120
build_weeks = 16
[[deployment.rungs]]
capacity = 45
cost = 200
build_weeks = 18
```

=== launch_services ===

content/rd_lines/launch_services.toml

```
# content/rd_lines/launch_services.toml
# 3 lines: propulsion/performance (how much you can lift) vs. reusability (the
# cost moat — Vector Dynamics' whole advantage) vs. reliability (the thing that
# turns a launch from a coin-flip into a contract-winning track record).
```

```
[propulsion]
id = "propulsion"
sub_industry = "launch_services"
name = "Propulsion & performance"
description = """Engine power and efficiency — how much mass you can throw to \
orbit and how far. The raw capability lever that decides which payload classes \
and orbits you can even bid on. Bigger, hotter, more efficient engines open \
bigger contracts."""
icon = "flame"
starting_level = 25
base_cost_per_quarter = 7
drives_specs = ["payload", "performance"]

[reusability]
id = "reusability"
sub_industry = "launch_services"
name = "Reusability"
description = """Recovering and reflying hardware instead of throwing it away. \
The structural cost moat — get it right and your cost-per-kilogram drops below \
what anyone tossing their rockets can match. Rivals can't copy it by trying \
harder, only by catching up on the same curve."""
icon = "recycle"
starting_level = 15
base_cost_per_quarter = 6
drives_specs = ["cost", "margin"]

[reliability]
id = "reliability"
sub_industry = "launch_services"
name = "Reliability"
description = """How consistently you reach orbit without losing the vehicle. \
Every successful flight compounds into trust; every failure grounds you and \
scares the manifest. This is what turns launches from binary gambles into a \
bankable track record customers pay a premium for."""
icon = "check-shield"
starting_level = 30
base_cost_per_quarter = 5
drives_specs = ["reliability", "performance"]
```

content/products/launch_services.toml

```
# content/products/launch_services.toml
# vehicles as products. The signature "launch" is the build/upgrade bet here.
# Note the workhorse feel: LOW decay (a proven vehicle flies for years) and
```

```
# faster build cadence than chips. A reliable rocket is a long-lived asset.

[small_lift]
id = "small_lift"
sub_industry = "launch_services"
name = "Small-lift vehicle"
tier = 1
description = """A dedicated small rocket for smallsats and rideshares — the \
Ascent Aerospace beachhead. Lower payload, but you fly it often, you fly it on \
the customer's schedule, and every clean flight builds the reliability record \
you'll need to climb the lift-class ladder."""
flavor_naming_hint = "e.g. 'Sprite', a small/fast vehicle name"

[small_lift.gates]
propulsion = 20
reliability = 25

[small_lift.economics]
build_cost = 35
build_weeks = 44
unit_margin = 0.30
capacity_type = "launch_capacity"
capacity_to_build = 1
capacity_to_run = 1
addressable_market = 800
ramp_weeks = 26
decay_per_quarter = 0.03        # a proven small-lift workhorse lasts

[small_lift.specs]
payload = 0.4
reliability = 0.4
cost = 0.2

[medium_lift]
id = "medium_lift"
sub_industry = "launch_services"
name = "Medium-lift reusable"
tier = 2
description = """The workhorse that makes the business — a reusable medium-lift \
vehicle that flies commercial constellations, government payloads, and anything \
in between. Reusability is the whole game here: crack it and your cost-per-kilo \
undercuts everyone; miss it and you're flying at a loss."""
flavor_naming_hint = "e.g. 'Vector-9', a workhorse vehicle name"

[medium_lift.gates]
propulsion = 45
reusability = 45
```

```
reliability = 45

[medium_lift.economics]
build_cost = 140
build_weeks = 78
unit_margin = 0.48
capacity_type = "launch_capacity"
capacity_to_build = 2
capacity_to_run = 2
addressable_market = 6500
ramp_weeks = 39
decay_per_quarter = 0.03        # the long-lived cash workhorse

[medium_lift.specs]
cost = 0.4
payload = 0.35
reliability = 0.25

[heavy_human]
id = "heavy_human"
sub_industry = "launch_services"
name = "Heavy-lift / human-rated"
tier = 3
description = """The flagship — heavy-lift capacity and the reliability bar to \
carry crew. The contracts are enormous and so is the prestige, but human-rating \
is the most demanding standard in the business: everything has to be near-perfect
and one failure is catastrophic in every sense."""
flavor_naming_hint = "e.g. 'Titan Heavy', a flagship vehicle name"

[heavy_human.gates]
propulsion = 70
reusability = 60
reliability = 70

[heavy_human.economics]
build_cost = 420
build_weeks = 130
unit_margin = 0.52
capacity_type = "launch_capacity"
capacity_to_build = 3
capacity_to_run = 2
addressable_market = 12000
ramp_weeks = 52
decay_per_quarter = 0.04        # flagship, but still a durable platform

[heavy_human.specs]
reliability = 0.45
```

```
payload = 0.35
performance = 0.20
```

content/capacity/launch_services.toml

```
# content/capacity/launch_services.toml
# one combined capacity type folding pads + vehicle fleet into "launch capacity"
# (kept to one type for clarity per the 1-2 guidance). starting_capacity is
# higher in tuning (3) so you can fly from day one.

[launch_capacity]
id = "launch_capacity"
sub_industry = "launch_services"
name = "Launch capacity"
unit_label = "slot"
description = """Your pads, integration facilities, and vehicle fleet — everythin
that lets you actually fly. Each active vehicle program and each booked launch \
draws on it. Demand past your capacity means a backlog, and a backlog means \
customers eyeing rivals who can fly them sooner."""

[[launch_capacity.rungs]]
capacity = 3
cost = 45
build_weeks = 40
[[launch_capacity.rungs]]
capacity = 6
cost = 110
build_weeks = 48
[[launch_capacity.rungs]]
capacity = 10
cost = 240
build_weeks = 56
[[launch_capacity.rungs]]
capacity = 15
cost = 460
build_weeks = 64
[[launch_capacity.rungs]]
capacity = 21
cost = 800
build_weeks = 72
[[launch_capacity.rungs]]
capacity = 28
cost = 1300
build_weeks = 80
```

=== satellite_constellations ===

content/rd_lines/satellite_constellations.toml

```
# content/rd_lines/satellite_constellations.toml
# 3 lines: satellite tech (capability per bird) vs. manufacturing (mass-productio
# cost — constellations live or die on per-unit cost) vs. network/ground (the
# system that turns a fleet into a service).

[satellite_tech]
id = "satellite_tech"
sub_industry = "satellite_constellations"
name = "Satellite technology"
description = """The capability packed into each bird — sensors, transponders, \
power, station-keeping. Better satellites mean more coverage, more bandwidth, or
sharper imagery per unit. The capability lever that decides what your \
constellation can actually sell."""
icon = "satellite"
starting_level = 30
base_cost_per_quarter = 6
drives_specs = ["capability", "coverage"]

[mass_production]
id = "mass_production"
sub_industry = "satellite_constellations"
name = "Mass production"
description = """Building satellites like products, not bespoke spacecraft. A \
constellation needs hundreds of them, so per-unit cost is everything — the \
difference between a viable network and a money pit. Driving manufacturing cost \
down is the unglamorous heart of the business."""
icon = "factory"
starting_level = 20
base_cost_per_quarter = 5
drives_specs = ["cost", "margin"]

[network]
id = "network"
sub_industry = "satellite_constellations"
name = "Network & ground systems"
description = """The inter-satellite links, ground stations, and software that \
turn a swarm of hardware into a service customers actually buy. Without it you \
have expensive metal in orbit; with it you have recurring revenue and a coverage
map that fills in."""
icon = "globe"
starting_level = 25
```

```
base_cost_per_quarter = 5
drives_specs = ["coverage", "reliability"]
```

content/products/satellite_constellations.toml

```
# content/products/satellite_constellations.toml
# constellations as products. Uses TWO capacity types: sat_manufacturing (to
# build/replenish the fleet) and ground_network (to serve it). Capital-heavy
# buildout, then recurring subscription revenue — moderate decay (fleets need
# refreshing but a deployed network earns steadily).

[comms_relay]
id = "comms_relay"
sub_industry = "satellite_constellations"
name = "Comms relay constellation"
tier = 1
description = """A modest fleet providing point-to-point connectivity — backhaul,
IoT, maritime. Your entry constellation: fewer birds, proven demand, and a way \
to get recurring revenue flowing while you learn to operate a network. Meridian \
Orbital started here before the capex hump."""
flavor_naming_hint = "e.g. 'LinkNet', a connectivity brand"

[comms_relay.gates]
satellite_tech = 25
network = 25

[comms_relay.economics]
build_cost = 90
build_weeks = 60
unit_margin = 0.40
capacity_type = "sat_manufacturing"
capacity_to_build = 2
capacity_to_run = 1
addressable_market = 1800
ramp_weeks = 39
decay_per_quarter = 0.05        # fleet needs refresh, but service is steady

[comms_relay.specs]
coverage = 0.4
capability = 0.35
cost = 0.25

[earth_observation]
id = "earth_observation"
sub_industry = "satellite_constellations"
```

```
name = "Earth-observation constellation"
tier = 2
description = """A fleet of imaging satellites selling a daily picture of the \
planet — agriculture, defense, climate, finance. Higher-value data, and the \
AI-analytics layer on top is where Lumen Skies pitches its story. Demands sharper
satellites and a network to move the imagery down."""
flavor_naming_hint = "e.g. 'Lumen-1', an imaging-fleet name"

[earth_observation.gates]
satellite_tech = 50
mass_production = 40
network = 45

[earth_observation.economics]
build_cost = 220
build_weeks = 100
unit_margin = 0.55
capacity_type = "sat_manufacturing"
capacity_to_build = 3
capacity_to_run = 2
addressable_market = 5500
ramp_weeks = 44
decay_per_quarter = 0.06

[earth_observation.specs]
capability = 0.45
coverage = 0.35
reliability = 0.20

[broadband_mega]
id = "broadband_mega"
sub_industry = "satellite_constellations"
name = "Broadband megaconstellation"
tier = 3
description = """The empire bet — hundreds of satellites delivering global \
broadband direct to customers. Staggering upfront capital and a manufacturing \
line that has to crank out birds like an assembly plant, but past the capex hump
it's a recurring-revenue machine with planetary reach."""
flavor_naming_hint = "e.g. 'Meridian Global', a broadband brand"

[broadband_mega.gates]
satellite_tech = 60
mass_production = 70
network = 65

[broadband_mega.economics]
build_cost = 700
```

```
build_weeks = 150
unit_margin = 0.58
capacity_type = "sat_manufacturing"
capacity_to_build = 5
capacity_to_run = 3
addressable_market = 20000
ramp_weeks = 60
decay_per_quarter = 0.07        # constant replenishment, but huge installed base

[broadband_mega.specs]
coverage = 0.45
cost = 0.30
capability = 0.25
```

content/capacity/satellite_constellations.toml

```
# content/capacity/satellite_constellations.toml
# TWO capacity types (the spec's suggested example): manufacturing throughput to
# BUILD the fleet, and ground network to SERVE it. Products consume manufacturing
# during the build bet; the engine can also draw ground_network for run capacity
# of large constellations. Both ladder so neither caps out.

[sat_manufacturing]
id = "sat_manufacturing"
sub_industry = "satellite_constellations"
name = "Satellite manufacturing"
unit_label = "line"
description = """Your assembly throughput — how many satellites you can build and
replenish per cycle. A constellation is never finished; birds age out and must be
replaced, so manufacturing is a permanent constraint, not a one-time build."""

[[sat_manufacturing.rungs]]
capacity = 3
cost = 50
build_weeks = 44
[[sat_manufacturing.rungs]]
capacity = 6
cost = 120
build_weeks = 52
[[sat_manufacturing.rungs]]
capacity = 10
cost = 260
build_weeks = 60
[[sat_manufacturing.rungs]]
capacity = 16
```

```
cost = 500
build_weeks = 70
[[sat_manufacturing.rungs]]
capacity = 23
cost = 880
build_weeks = 80
[[sat_manufacturing.rungs]]
capacity = 32
cost = 1450
build_weeks = 90

[ground_network]
id = "ground_network"
sub_industry = "satellite_constellations"
name = "Ground network"
unit_label = "station"
description = """The ground stations and network operations that bring your fleet
signal down to paying customers. Coverage and capacity here gate how much of your
orbital fleet you can actually monetize — birds you can't talk to don't earn."""

[[ground_network.rungs]]
capacity = 2
cost = 20
build_weeks = 24
[[ground_network.rungs]]
capacity = 5
cost = 55
build_weeks = 30
[[ground_network.rungs]]
capacity = 9
cost = 120
build_weeks = 36
[[ground_network.rungs]]
capacity = 14
cost = 230
build_weeks = 42
[[ground_network.rungs]]
capacity = 20
cost = 400
build_weeks = 48
[[ground_network.rungs]]
capacity = 28
cost = 660
build_weeks = 54
```

=== space_stations ===

content/rd_lines/space_stations.toml

```
# content/rd_lines/space_stations.toml
# 3 lines: module/structures (what you can build in orbit) vs. life-support
# (the human-rating bar that unlocks tourism/long-duration) vs. on-orbit assembly
# (how cheaply and reliably you can put it all together up there).

[module_tech]
id = "module_tech"
sub_industry = "space_stations"
name = "Module & structures"
description = """The habitats, labs, and docking structures themselves — volume,
durability, what they can house. Better module tech means more usable space and \
more kinds of tenant you can serve, from research racks to tourist suites."""
icon = "module"
starting_level = 30
base_cost_per_quarter = 6
drives_specs = ["capacity", "capability"]

[life_support]
id = "life_support"
sub_industry = "space_stations"
name = "Life support & habitation"
description = """Keeping people alive and comfortable in orbit — air, water, \
radiation, the works. This is the human-rating bar: clear it and you unlock \
tourism and long-duration crews; fall short and you're limited to cargo and \
short visits. The hardest, most safety-critical line."""
icon = "heart-pulse"
starting_level = 20
base_cost_per_quarter = 5
drives_specs = ["safety", "capability"]

[assembly]
id = "assembly"
sub_industry = "space_stations"
name = "On-orbit assembly"
description = """How efficiently you build and expand in space — autonomous \
docking, robotic construction, in-orbit fit-out. It drives the cost and speed of
standing up modules, which is the difference between a station that pencils out \
and one that bankrupts you on assembly alone."""
icon = "robot-arm"
starting_level = 25
base_cost_per_quarter = 5
```

```
drives_specs = ["cost", "reliability"]
```

content/products/space_stations.toml

```
# content/products/space_stations.toml
# stations/modules as products. Long, expensive builds; low decay (a station is
# infrastructure that lasts), but lumpy tenant-driven revenue is expressed via
# slower ramps and the tuning's share volatility. Apex Station is the anchor flav

[research_module]
id = "research_module"
sub_industry = "space_stations"
name = "Research module"
tier = 1
description = """A single pressurized module leased to researchers and in-space \
manufacturing — microgravity labs, materials experiments, pharma. Your beachhead:
one module, a handful of anchor tenants, and proof that commercial orbit pays. \
Apex Station started exactly here."""
flavor_naming_hint = "e.g. 'Lab-1', a research-module name"

[research_module.gates]
module_tech = 25
assembly = 25

[research_module.economics]
build_cost = 110
build_weeks = 78
unit_margin = 0.44
capacity_type = "module_construction"
capacity_to_build = 2
capacity_to_run = 1
addressable_market = 1400
ramp_weeks = 44                 # lumpy tenant signing — slow to fill
decay_per_quarter = 0.03        # infrastructure: long-lived

[research_module.specs]
capability = 0.45
capacity = 0.35
cost = 0.20

[tourism_habitat]
id = "tourism_habitat"
sub_industry = "space_stations"
name = "Tourism habitat"
tier = 2
```

```
description = """A human-rated habitat for orbital tourism and private astronaut
stays — the headline-grabbing, high-margin tenant class. It demands real \
life-support maturity (this is where human-rating bites), but a suite with a view
of Earth commands prices nothing on the ground can touch."""
flavor_naming_hint = "e.g. 'Horizon Suites', a hospitality-in-orbit name"

[tourism_habitat.gates]
module_tech = 45
life_support = 55
assembly = 40

[tourism_habitat.economics]
build_cost = 260
build_weeks = 110
unit_margin = 0.58
capacity_type = "module_construction"
capacity_to_build = 3
capacity_to_run = 2
addressable_market = 4200
ramp_weeks = 52
decay_per_quarter = 0.04

[tourism_habitat.specs]
safety = 0.4
capability = 0.35
capacity = 0.25

[full_station]
id = "full_station"
sub_industry = "space_stations"
name = "Full commercial station"
tier = 3
description = """A complete multi-module station — research, habitation, \
manufacturing, and docking under one roof, leasing space at scale. The \
empire-in-orbit bet: enormous capital, years of on-orbit assembly, and a \
real-estate-in-space model where you're the landlord of a destination, not just \
a module."""
flavor_naming_hint = "e.g. 'Apex One', a flagship station name"

[full_station.gates]
module_tech = 65
life_support = 65
assembly = 70

[full_station.economics]
build_cost = 750
build_weeks = 170
```

```
unit_margin = 0.55
capacity_type = "module_construction"
capacity_to_build = 5
capacity_to_run = 3
addressable_market = 14000
ramp_weeks = 65
decay_per_quarter = 0.03        # a station is a generational asset

[full_station.specs]
capacity = 0.4
capability = 0.35
safety = 0.25
```

content/capacity/space_stations.toml

```
# content/capacity/space_stations.toml
# one capacity type (module construction / berths). The slowest, most capital-
# heavy buildout of all six sub-industries — building in orbit is brutal. Rungs
# ladder high so even a multi-station operator has a next rung.

[module_construction]
id = "module_construction"
sub_industry = "space_stations"
name = "Module construction"
unit_label = "berth"
description = """Your in-orbit construction and berthing capacity — the assembly
slots, robotic systems, and docking ports that let you stand up and operate \
modules. Building in space is the hardest, slowest capacity to add, and every \
active module program and live station draws on it."""

[[module_construction.rungs]]
capacity = 3
cost = 70
build_weeks = 60
[[module_construction.rungs]]
capacity = 6
cost = 160
build_weeks = 70
[[module_construction.rungs]]
capacity = 10
cost = 340
build_weeks = 82
[[module_construction.rungs]]
capacity = 15
cost = 640
```

```
build_weeks = 94
[[module_construction.rungs]]
capacity = 21
cost = 1100
build_weeks = 106
[[module_construction.rungs]]
capacity = 28
cost = 1800
build_weeks = 118
```

=== Per-industry tuning (all six) ===

content/products/_tuning.toml

```
# content/products/_tuning.toml
# One block per sub-industry. These knobs make each industry's economics and
# pace feel distinct without touching the engine. Tuned AGAINST each other so no
# sub-industry is strictly easiest — each trades ease in one axis for difficulty
# in another.
#
# Quick read of the intended contrast:
#   frontier_model_lab → fast / cheap / many-parallel / hype-volatile (high churn
#   vertical_ai_saas   → cheapest+fastest builds / very sticky / capacity-light
#   ai_chips           → slow / expensive / few-parallel / sticky design-wins
#   launch_services    → fast cadence / workhorse decay / very contract-sticky
#   satellite_constel. → capital-heavy buildout / moderate / two-capacity juggle
#   space_stations     → slowest / most expensive / lumpy revenue / fewest bets

[frontier_model_lab]
starting_capacity = 2
rd_diminishing_k = 0.7          # returns diminish gently — scaling keeps paying
frontier_pull = 1.3             # strong catch-up aid (the field moves together)
max_concurrent_bets = 4         # many parallel training runs
build_cost_mult = 1.0
build_time_mult = 1.0           # the baseline-fast reference
decay_mult = 1.0                # treadmill is real (per-product decay already hi
share_volatility = 0.25         # hype-driven — share swings fast

[vertical_ai_saas]
starting_capacity = 4           # capacity-light: you start with room
rd_diminishing_k = 0.65         # domain depth keeps compounding
frontier_pull = 0.8             # HARD AXIS: little catch-up aid — entrenched
                                # incumbents (Lexiq-style) are sticky and slow to
                                # dislodge. SaaS difficulty is winning share, not
```

```
max_concurrent_bets = 3         # focused: a vertical wins by depth, not breadth
build_cost_mult = 0.55          # cheapest builds of all six
build_time_mult = 0.6           # and the fastest
decay_mult = 0.7                # subscriptions are sticky — slow churn
share_volatility = 0.10         # retention-driven — share barely moves either wa

[ai_chips]
starting_capacity = 2
rd_diminishing_k = 0.8          # the node gets brutally hard at the frontier
frontier_pull = 1.2
max_concurrent_bets = 3         # fewer parallel tape-outs — they're huge
build_cost_mult = 1.4           # capital-heavy
build_time_mult = 1.6           # and slow
decay_mult = 1.1                # frontier parts obsolesce fast
share_volatility = 0.15         # stickier, design-win-driven

[launch_services]
starting_capacity = 3           # you can fly from day one
rd_diminishing_k = 0.75
frontier_pull = 1.1
max_concurrent_bets = 3         # HARD AXIS: vehicle programs are large and
                                # capital-heavy — you can't run many at once,
                                # and a launch failure (events layer) sets you ba
build_cost_mult = 1.2           # capital-heavy vehicles
build_time_mult = 0.7           # but faster cadence than tape-outs once flying
decay_mult = 0.6                # a reliable vehicle is a long-lived workhorse
share_volatility = 0.12         # contract/reliability-driven, very sticky

[satellite_constellations]
starting_capacity = 3
rd_diminishing_k = 0.78
frontier_pull = 1.15
max_concurrent_bets = 3         # constellations are big, capital-heavy bets
build_cost_mult = 1.3           # capital-heavy buildout
build_time_mult = 1.2           # long deployment timelines
decay_mult = 0.8                # fleets refresh, but installed base earns steadi
share_volatility = 0.16         # mixed: comms sticky, imaging more hype-exposed

[space_stations]
starting_capacity = 2
rd_diminishing_k = 0.82         # building in orbit gets very hard at the top
frontier_pull = 1.05            # few players, little shared frontier to catch
max_concurrent_bets = 2         # the fewest parallel bets — each is colossal
build_cost_mult = 1.5           # the most expensive builds of all six
build_time_mult = 1.7           # and the slowest
decay_mult = 0.5                # stations are generational infrastructure
share_volatility = 0.18         # lumpy, tenant-driven revenue
```